
Medlytics Hypothyroidism Group 8

By: Ananya, Aryan, Sourya

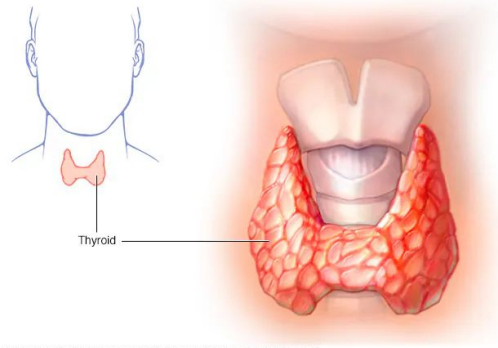
Background

Problem?

Nearly 5% of Americans over age 12 are affected by hypothyroidism, an endocrine disorder in which the thyroid gland fails to produce enough thyroid hormones. Hypothyroidism can lead to symptoms like fatigue, weight gain, cold tolerance, or depression.

Accurate diagnosis is possible through blood tests like TSH and T4, but collecting patient data across numerous features is time-consuming and expensive. Scalable diagnosis tools that are feasible for low-resource areas is needed

Our goal: Build and test multiple machine learning models to identify their accuracy in predicting hypothyroidism





Data Cleaning Steps

- 1) Replaced all ? placeholders with NaN to mark missing data
- 2) Converted columns to the data types we needed
 - a) Converted numeric columns into float64
 - b) Separated numeric and categorical columns
- 3) Dropped columns with no informative value:
 - a) TBG, TBG Measured, and others with only one unique value
- 4) Cleaned Class column to remove irrelevant parts and convert
 - a) “Negative” became 0
 - b) All types of “hypothyroid” became 1

Model Performance Results

Model Performance



	Test Accuracy	Test Balanced Accuracy
Decision Tree (age, sex)	92%	52%
Decision Tree (all features)	99.2%	93 %
Decision Tree (top 5 features)	99.5%	95.1%
MLP (all features)	97.1%	91.1%
MLP (all features, hidden layers)	97.4%	97.1%

Decision Tree (age, sex)

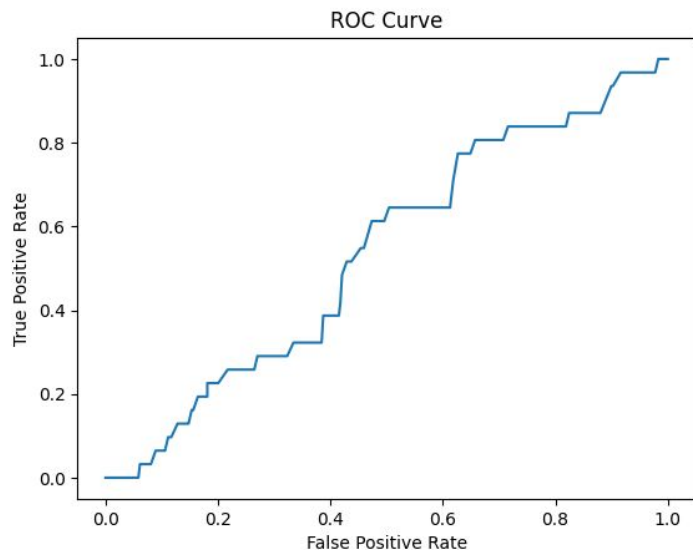


Accuracy: 92% (matched zero rule classification)

Balanced Accuracy: 52%

Confusion Matrix

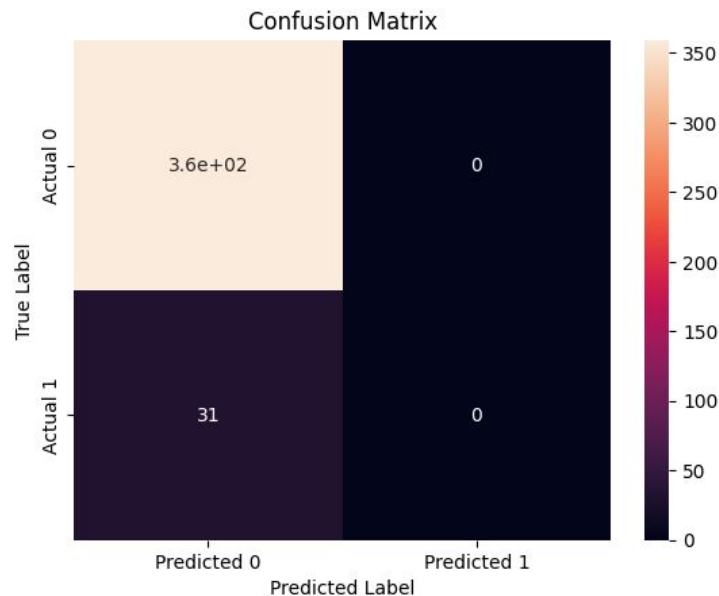
ROC Curve



```
X = dataset[["Age", "Sex"]]
```

```
y = dataset["Class"]
```

```
decision tree = DecisionTreeClassifier(random state=0)
```

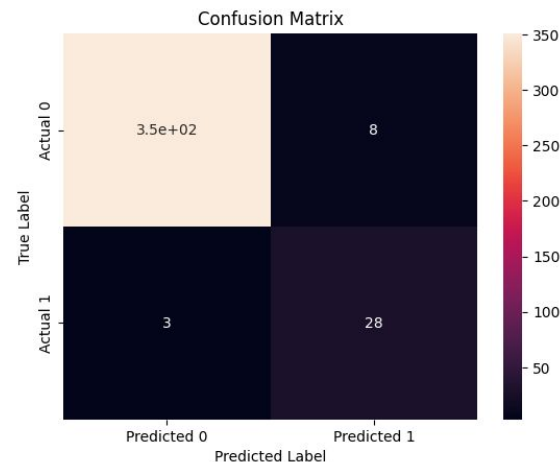


Improving Decision Tree

- To improve decision tree performance, we used feature engineering to only select the most important features
- Identified top 5 features as TSH, FTI, TT4, T3, and T4U (trained a random forest and identified top 5 predictive features)
- By reducing the number of features, we minimized noise and decreased the risk of overfitting

TSH	0.707934
FTI	0.080488
TT4	0.071929
T3	0.054101
T4U	0.024246
Age	0.020819
On Thyroxine	0.015149
Sex	0.006860
Thyroid Surgery	0.003855
Sick	0.003046
dtype:	float64

Achieved a balanced accuracy of 95%



Decision Tree (all features)

Accuracy: 99.2%

Balanced Accuracy: 93%

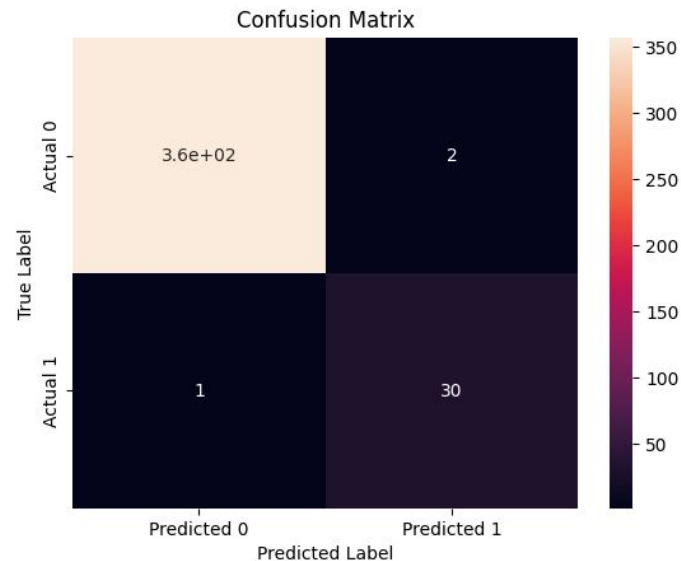
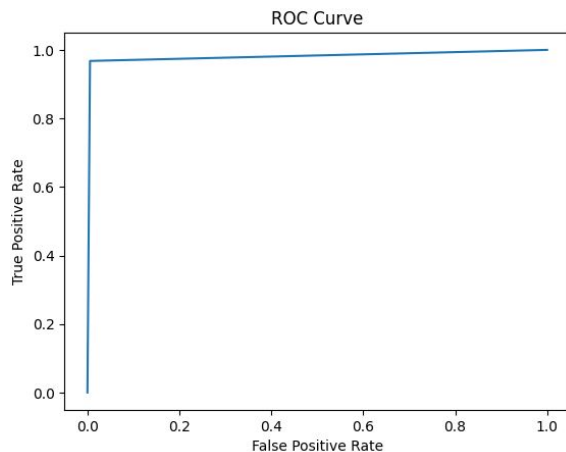
Confusion Matrix

ROC Curve

```
X = dataset[numeric columns + categorical columns]
```

```
y = dataset['Class']
```

```
decision tree = DecisionTreeClassifier(random state=0)
```



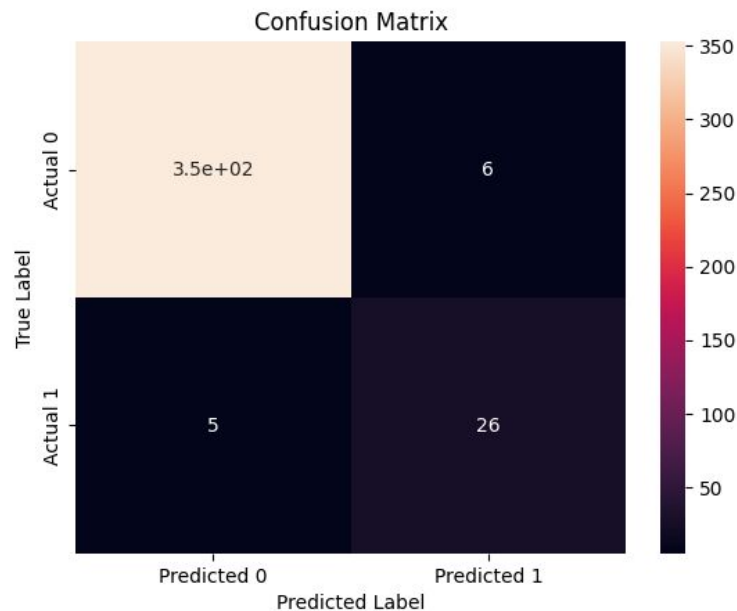
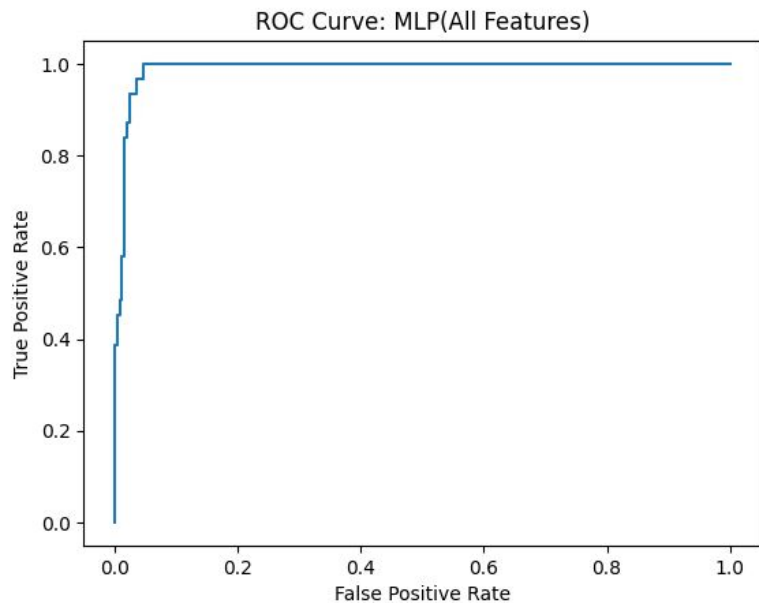
MLP (all features)



Accuracy: 95.9%

Balanced Accuracy: 91.1%

```
MLPClassifier(solver='lbfgs', alpha = 0.0005, random_state=5, max_iter=150)
```

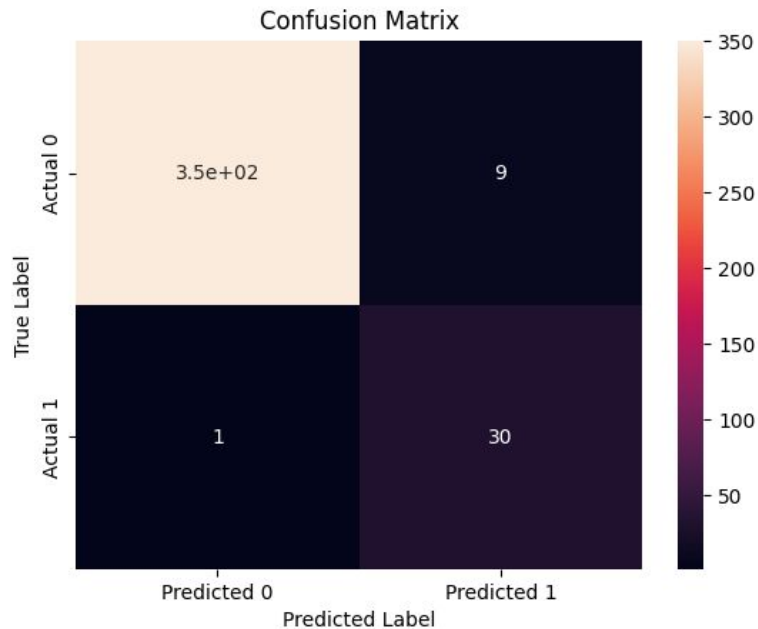
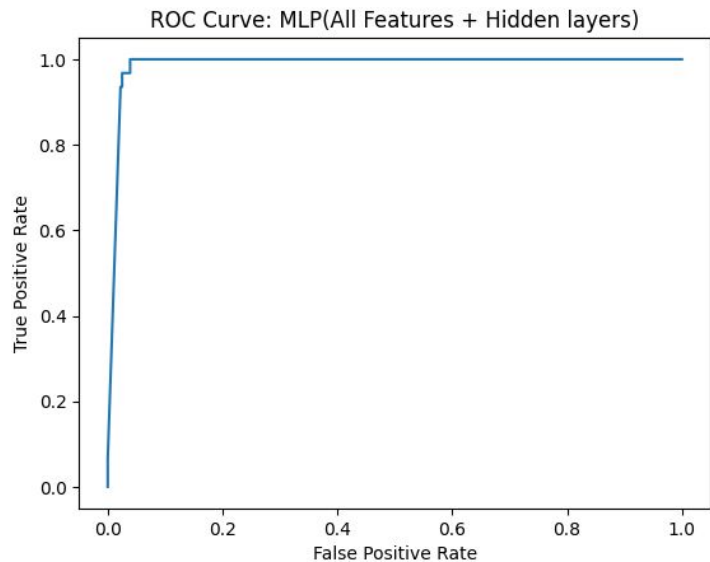


MLP (all features, hidden layers)

Accuracy: 97.4%

Balanced Accuracy: 97.1%

```
MLPClassifier(solver='lbfgs', alpha = 0.0005,  
hidden_layer_sizes=(5,5),random_state=5, max_iter=150)
```



Expansions and Improvements



Challenges

- Class imbalance
 - Decided to use balanced accuracy for a more accurate statistic
- `.get_dummies()` vs `.replace()`
 - `Get_Dummies` separates a categorical feature into multiple columns
 - We found the increased dimensionality to be harder to work with, and we just decided to replace the values using `.replace`



Improvements

- When data processing, dropping all rows with NaNs results in a lot of data lost (~30%)
 - For numerical columns, can instead try replacing missing values with mean or median
- Move beyond just binary classification:
 - Train a model to distinguish between primary, secondary, and compensated hypothyroidism
- Class weighting in the loss function, penalize false negatives more
- Try model ensembling for better performance
- Use the pairwise correlation to eliminate features